

# Design Document - Machine Problem 2.2

Student Name: Yash Phatak

## 1 Introduction

The project aims is to enhance the existing Tiny Social Network Service (SNS) from MP2.1 by integrating features that ensure fault tolerance, high availability, and scalability. These features allow the system to handle failures transparently while maintaining uninterrupted service for users. The architecture introduces Master-Slave redundancy, periodic synchronization, and inter-process communication mechanisms to support distributed operations. It incorporates gRPC and RabbitMQ for reliable communication and asynchronous updates, ensuring data consistency and seamless user experience in a highly distributed environment.

## 2 System Components

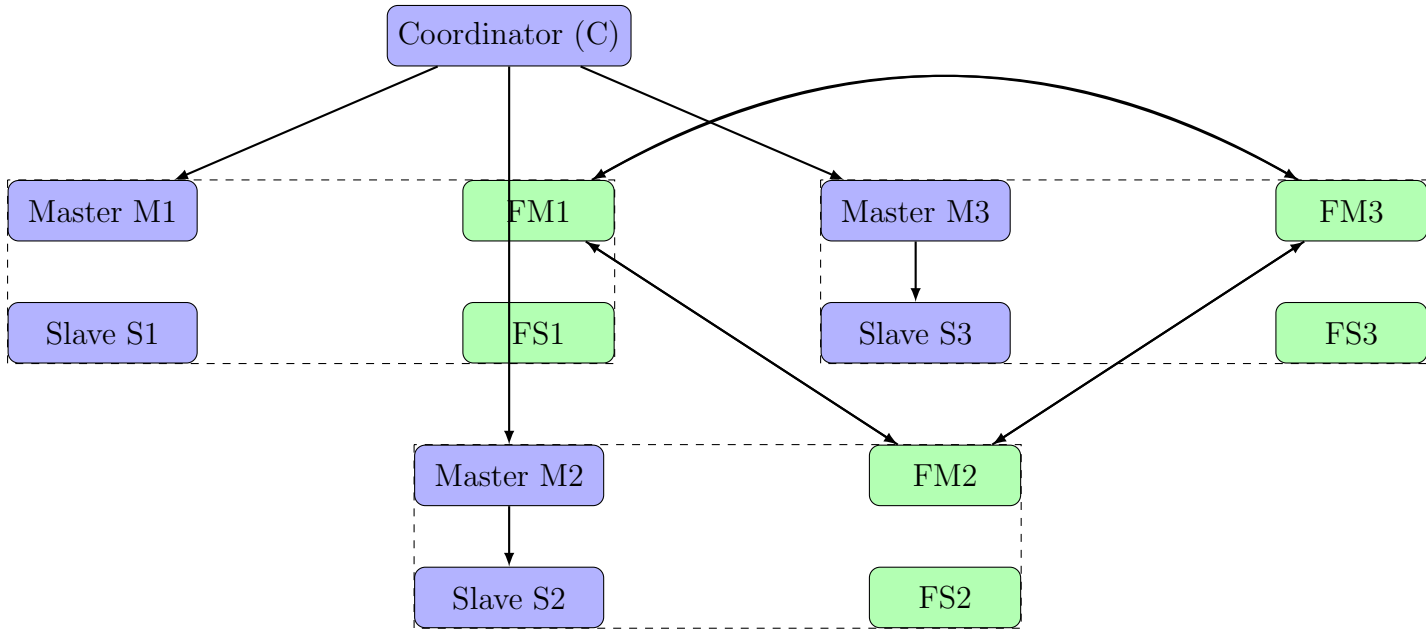


Figure 1: System Components

The system follows a client-server model with a coordinator and synchronizer managing the interaction between clients and servers. The client communicates with the coordinator, which then relays requests to the appropriate server. The synchronizer is responsible for managing inter-cluster communication, ensuring that the data across different clusters is synchronized and consistent.

## 2.1 Coordinator

- **File:** `coordinator.cc`
- **Description:** The Coordinator handles communication between the client and master and slave servers by keeping track of clusters and their server IDs. It sends client requests to the correct server by providing the necessary server information. The Coordinator also keeps a list of Synchronizers to manage synchronization between different clusters, ensuring data remains consistent across them. information.

## 2.2 Client Program

- **File:** `tsc.cc`
- **Description:** The client sends requests to the Coordinator, which processes them and provides the client with the necessary server information. The client then uses this information to send requests directly to the servers and receive responses. Additionally, the Client periodically sends heartbeat signals to the servers.

## 2.3 Server Program

- **File:** `tsd.cc`
- **Description:** The server registers with the Coordinator to establish a communication link with the client. It handles requests from the client, processes them, and sends back the appropriate responses. The master server forwards the requests to the slave server for fault tolerance.

## 2.4 Synchronizer Program

- **File:** `tsd.cc`
- **Description:** The Synchronizer registers with the Coordinator by sending a heartbeat. It then uses RabbitMQ to publish and consume messages between clusters, ensuring data consistency across the system.

# 3 Implementation

## 3.1 Replication of Master and Slave Information

To ensure that the slave can handle requests if the master crashes, replication of master and slave data is implemented. This is done by creating a new gRPC stub and retrieving slave information from the coordinator. The request is then forwarded to the slave server, which can take over in case of a failure of the master.

## 3.2 Master Failure and Client Reconnection

In the event of a master crash, when the client attempts to reconnect, the coordinator provides the client with information about the slave server. This ensures that the client can reconnect to the slave instead of the crashed master server, maintaining uninterrupted service.

### 3.3 Client-Server Heartbeat Mechanism

A heartbeat mechanism is implemented between the client and server to monitor connection status. If the client crashes or loses connection, the server sets the connection status to false. When the client attempts to reconnect, it is allowed back into the system by the server, ensuring fault tolerance and continuous communication.

### 3.4 Synchronizer Registration and Coordination

The synchronizer registers itself with the coordinator by sending a heartbeat. The coordinator maintains a list of active synchronizers and uses this list to ensure proper coordination. The synchronizer then uses RabbitMQ for message publishing and consumption to maintain consistent data and synchronize information across clusters.

### 3.5 Synchronizer Management and Cluster Coordination

The **Master Synchronizer** is responsible for managing the state of the clusters and ensuring that they are in sync. It maintains a list of active synchronizers and coordinates the flow of information between the clusters. The synchronizers are assigned to their respective clusters using unique identifiers, such as  $(\text{syncId}-1)\%3 + 1$ , to help ensure proper synchronization between the clusters.

### 3.6 Message Publishing and Consumption Using RabbitMQ

To keep the system in sync and ensure that data flows seamlessly between the clusters, RabbitMQ is used to publish and consume messages. Synchronizers communicate through RabbitMQ by publishing synchronization messages to specific queues. The messages contain data updates and synchronization updates for other clusters. Each synchronizer consumes messages from the appropriate queues to ensure that all clusters are kept up to date and consistent.

## 4 Execution

### 4.1 Commands and Usage

- **Command for running the Coordinator:** `./coordinator -p {port-no}`
- **Command for running the Client:** `./tsc -h {hostname} -k {port-no} -u {username}`
- **Command for running the Server:** `./tsd -c {cluster id} -s {server id} -h {coordinator-hostname} -k {coordinator-port-no} -p {server-port}`
- **Command for running the Synchronizer:** `./synchronizer -h {coordinatorIP} -k {coordinatorPort} -p portNumber -i synchronizerID`

### 4.2 Test Cases

The screenshots for each testcase are attached in the Screenshots section

- **Test Case 1: Sanity Check**
- **Commands Used:**  
Start the coordinator: `./coordinator -p 9090`  
Start server with id 1 and cluster id 1 : `./tsd -c 1 -s 1 -h localhost -k 9090 -p`

10000

Start server with id 2 and cluster id 1 : `./tsd -c 1 -s 2 -h localhost -k 9090 -p 10001`

Login with client 1: `./tsc -h localhost -k 9090 -u 1`

List command: `list`

Timeline command: `timeline`

Similarly, other commands are executed for user 4 as shown in the screenshot below.

- **Test Case 2 : List/Follow/Timeline**

- **Commands Used:**

Start the coordinator: `./coordinator -p 9090`

Start 6 servers i.e. 3 master servers and 3 slave servers for clusters 1, 2 and 3 using the commands in section 5.1.

Start 6 synchronizers i.e. for clusters 1, 2 and 3 using the commands in section 4.1.

Login for users 1,2 and 3

List command: `list`

Follow command: `follow {username}`

Timeline command: `timeline`

The commands executed are shown in the screenshots below in section 4.3.

- **Test Case 3: Check resilience if the master fails**

- **Commands Used:**

Initially, The same commands are executed as in testcase 2.

After that, users 1,2 and 3 are crashed.

The master server of cluster 2 is crashed.

Login for users 1,2,3 and 5.

List command: `list`

Follow command: `follow {username}`

Timeline command: `timeline`

The commands executed are shown in the screenshots below in section 4.3.

## 4.3 Screenshots

```
csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 1
Logging Initialized. Client starting...0.0.0.0 10000

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> list
Command completed successfully
All users: 1, 4,
Followers:
Cmd> follow 4
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 4,
Followers: 4,
Cmd> timeline
Command completed successfully
Now you are in the timeline
p11
p12
4 (Sun Nov 24 14:29:12 2024) >> p41
4 (Sun Nov 24 14:29:13 2024) >> p42
```

Figure 2: Testcase 2 - User 1

```
csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 4
Logging Initialized. Client starting...0.0.0.0 10000

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> list
Command completed successfully
All users: 1, 4,
Followers:
Cmd> follow 1
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 4,
Followers: 1,
Cmd> timeline
Command completed successfully
Now you are in the timeline
1 (Sun Nov 24 14:28:56 2024) >> p11
1 (Sun Nov 24 14:29:00 2024) >> p12

p41
p42
```

Figure 3: Testcase 2 - User 4

```
csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 1
Logging Initialized. Client starting...0.0.0.0 10000
```

```
===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
```

```
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers:
Cmd> follow 2
Command completed successfully
Cmd> follow 3
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers: 2, 3,
Cmd> timeline
Command completed successfully
Now you are in the timeline
p11
p12
2 (Sun Nov 24 15:24:21 2024) >> p21
2 (Sun Nov 24 15:24:23 2024) >> p22
3 (Sun Nov 24 15:25:15 2024) >> p31
3 (Sun Nov 24 15:25:17 2024) >> p32
```

Figure 4: Testcase 2 - User 1

```

csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 2
Logging Initialized. Client starting...0.0.0.0 20000

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====

Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers:
Cmd> follow 1
Command completed successfully
Cmd> follow 3
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3,
Followers: 1, 3,
Cmd> timeline
Command completed successfully
Now you are in the timeline
1 (Sun Nov 24 15:22:44 2024) >> p11

1 (Sun Nov 24 15:22:44 2024) >> p12

p21
p22
3 (Sun Nov 24 15:25:15 2024) >> p31

3 (Sun Nov 24 15:25:17 2024) >> p32

```

Figure 5: Testcase 2 - User 2

```
csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 3
Logging Initialized. Client starting...0.0.0.0 30000
```

```
===== TINY SNS CLIENT =====
```

```
Command Lists and Format:
```

```
FOLLOW <username>
```

```
UNFOLLOW <username>
```

```
LIST
```

```
TIMELINE
```

```
=====
```

```
Cmd> list
```

```
Command completed successfully
```

```
All users: 1, 2, 3,
```

```
Followers:
```

```
Cmd> follow 1
```

```
Command completed successfully
```

```
Cmd> follow 2
```

```
Command completed successfully
```

```
Cmd> list
```

```
Command completed successfully
```

```
All users: 1, 2, 3,
```

```
Followers: 1, 2,
```

```
Cmd> timeline
```

```
Command completed successfully
```

```
Now you are in the timeline
```

```
1 (Sun Nov 24 15:22:44 2024) >> p11
```

```
1 (Sun Nov 24 15:22:44 2024) >> p12
```

```
2 (Sun Nov 24 15:24:21 2024) >> p21
```

```
2 (Sun Nov 24 15:24:23 2024) >> p22
```

```
p31
```

```
p32
```

```
□
```

Figure 6: Testcase 2 - User 3



```
csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 1
Logging Initialized. Client starting...0.0.0.0 10000
```

```
===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
```

```
Cmd> follow 5
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3, 5,
Followers: 2, 3, 5,
Cmd> timeline
Command completed successfully
Now you are in the timeline
2 (Wed Nov 27 11:54:53 2024) >> p21

2 (Wed Nov 27 11:54:54 2024) >> p22

3 (Wed Nov 27 11:55:16 2024) >> p31

3 (Wed Nov 27 11:55:17 2024) >> p32

5 (Wed Nov 27 12:00:13 2024) >> p51

5 (Wed Nov 27 12:00:14 2024) >> p52

3 (Wed Nov 27 12:01:32 2024) >> p33

3 (Wed Nov 27 12:01:34 2024) >> p34

2 (Wed Nov 27 12:03:00 2024) >> p23

2 (Wed Nov 27 12:03:02 2024) >> p24
```

```
p13
p14
```

Figure 7: Testcase 3 - User 1

```
csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 2
Logging Initialized. Client starting...0.0.0.0 20001

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> follow 5
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3, 5,
Followers: 1, 3, 5,
Cmd> timeline
Command completed successfully
Now you are in the timeline
1 (Wed Nov 27 11:54:31 2024) >> p11

1 (Wed Nov 27 11:54:32 2024) >> p12

3 (Wed Nov 27 11:55:16 2024) >> p31

3 (Wed Nov 27 11:55:17 2024) >> p32

5 (Wed Nov 27 12:00:13 2024) >> p51

5 (Wed Nov 27 12:00:14 2024) >> p52

3 (Wed Nov 27 12:01:32 2024) >> p33

3 (Wed Nov 27 12:01:34 2024) >> p34

p23
p24
1 (Wed Nov 27 12:05:06 2024) >> p13

1 (Wed Nov 27 12:05:08 2024) >> p14
```

Figure 8: Testcase 3 - User 2

```
csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 3
Logging Initialized. Client starting...0.0.0.0 30000
```

```
===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> follow 5
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3, 5,
Followers: 1, 2, 5,
Cmd> timeline
Command completed successfully
Now you are in the timeline
1 (Wed Nov 27 11:54:31 2024) >> p11

1 (Wed Nov 27 11:54:32 2024) >> p12

2 (Wed Nov 27 11:54:53 2024) >> p21

2 (Wed Nov 27 11:54:54 2024) >> p22

5 (Wed Nov 27 12:00:13 2024) >> p51

5 (Wed Nov 27 12:00:14 2024) >> p52

p33
p34
2 (Wed Nov 27 12:03:00 2024) >> p23

2 (Wed Nov 27 12:03:02 2024) >> p24

1 (Wed Nov 27 12:05:06 2024) >> p13

1 (Wed Nov 27 12:05:08 2024) >> p14
```

Figure 9: Testcase 3 - User 3

```
csce662@yash:~/mp2_source_code$ ./tsc -h localhost -k 9000 -u 5
Logging Initialized. Client starting...0.0.0.0 20001
```

```
===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> list
Command completed successfully
All users: 1, 2, 3, 5,
Followers:
Cmd> follow 1
Command completed successfully
Cmd> follow 2
Command completed successfully
Cmd> follow 3
Command completed successfully
Cmd> list
Command completed successfully
All users: 1, 2, 3, 5,
Followers: 1, 2, 3,
Cmd> timeline
Command completed successfully
Now you are in the timeline
p51
p52
3 (Wed Nov 27 12:01:32 2024) >> p33
3 (Wed Nov 27 12:01:34 2024) >> p34
2 (Wed Nov 27 12:03:00 2024) >> p23
2 (Wed Nov 27 12:03:02 2024) >> p24
1 (Wed Nov 27 12:05:06 2024) >> p13
1 (Wed Nov 27 12:05:08 2024) >> p14
```

Figure 10: Testcase 3 - User 5